

Module 5: Testing, Quality & Maintenance

Black-Box (ECP/BVA), White-Box (Basis Path & Cyclomatic Complexity), Mutation & 4 Maintenance Types

Module V: Software Testing Strategies, Quality Assurance & Maintenance

1. Verification vs. Validation (Boehm: "Building product right" vs. "Building right product")
2. Black-Box Testing: Equivalence Class Partitioning (ECP) & Boundary Value Analysis (BVA)
3. White-Box (Structural) Testing: Control Flow Graphs (CFG) & Logic Path Coverage
4. McCabe's Cyclomatic Complexity ($V(G) = E - N + 2P = P + 1$) & Independent Basis Paths
5. Advanced Structural Coverage: Branch, Condition, Multiple Condition & MC/DC
6. Testing Levels Hierarchy: Unit Testing (Stubs & Drivers), Integration & System Testing
7. Integration Strategies: Big-Bang, Top-Down (Depth/Breadth-First) vs. Bottom-Up Sandwich
8. Acceptance Testing (Alpha vs. Beta Testing) & Regression Testing Test-Suite Minimization
9. Mutation Testing: Generating Mutants, Killed Mutants (D) & Mutation Score (MS)
10. Software Quality Assurance: ISO 9126, McCall's Quality Factors & CMMI 5 Levels
11. Software Maintenance: 4 Classical Categories (Corrective, Adaptive, Perfective, Preventive)
12. Comprehensive Solved BIT Mesra & GATE Exam Question Bank (8 Questions)

Topic 2: Black-Box (Functional) Testing Techniques

Testing Method	Test Case Derivation Principle	Example for Input Range [1, 100]
1. Equivalence Class Partitioning (ECP)	Divides input domain into valid and invalid equivalence classes. One test case is selected to represent each class.	- Valid Class: [1, 100] (e.g., 50) - Invalid Class 1: < 1 (e.g., -5) - Invalid Class 2: > 100 (e.g., 150)
2. Boundary Value Analysis (BVA)	Selects test cases at the extreme boundaries and just inside/outside boundaries ($Min, Min+, Nom, Max-, Max$).	Test values: 1 (Min), 2 (Min+), 50 (Nom), 99 (Max-), 100 (Max), plus invalid boundaries 0 and 101.

Topic 3 & 4: White-Box Testing & McCabe's Cyclomatic Complexity

McCabe's Cyclomatic Complexity ($V(G)$) Formulas: Let G be the Control Flow Graph of a program: 1. **Formula 1 (Edges & Nodes):**

$$V(G) = E - N + 2P$$

Where E is number of flow graph edges, N is number of nodes, and P is number of connected components ($P = 1$ for single programs). 2. **Formula 2 (Predicate Nodes):**

$$V(G) = P + 1$$

Where P is the number of binary decision/conditional predicate nodes ('if', 'while', 'for'). 3. **Formula 3 (Enclosed Regions):**

$$V(G) = \text{Number of Enclosed Planar Regions} + 1 \text{ (Outer Region)}$$

Step-by-Step Solved Problem: Cyclomatic Complexity & Basis Paths Calculation

Code Snippet:

```
1: if (a > 10) {  
2:     if (b < 20)  
3:         x = 1;  
4:     else  
5:         x = 2;  
6: } else {  
7:     x = 3;  
8: }  
9: return x;
```

1. **Predicate Nodes (P):** Two conditional tests (Node 1: `a > 10`, Node 2: `b < 20`).
2. **Cyclomatic Complexity:** $V(G) = P + 1 = 2 + 1 = 3$.
3. **3 Linearly Independent Basis Paths:** - *Path 1:* 1 → 6 → 7 → 9 (Condition 1 is False). - *Path 2:* 1 → 2 → 3 → 9 (Condition 1 is True, Condition 2 is True). - *Path 3:* 1 → 2 → 5 → 9 (Condition 1 is True, Condition 2 is False).

Topic 9: Mutation Testing & Quality Evaluation

Mutation Score (MS) Formula:

$$\text{Mutation Score (MS)} = \frac{D}{T - E} \times 100\%$$

Where T is total generated mutants, D is number of killed (detected) mutants, and E is number of equivalent mutants (mutants that have identical functional behavior to original program).

Topic 11: Software Maintenance Categories

Maintenance Type	Purpose & Triggering Factor	Typical Effort Share
1. Corrective	Fixing residual latent software defects and bugs discovered in production.	≈ 20%
2. Adaptive	Modifying software to accommodate changes in external environment (new OS, hardware upgrade, legal regulations).	≈ 25%
3. Perfective	Enhancing software functionality, improving performance, and optimizing code maintainability based on user requests.	≈ 50% (Dominant)
4. Preventive	Refactoring and restructuring code to forestall future potential failures (Software Re-engineering).	≈ 5%



Top BIT Mesra Exam Questions & Answers (Module V)

Q1. Compare Stubs and Drivers in Integration Testing with diagrams. (8 Marks)

- **Driver:** A dummy calling program written to invoke and pass test data to the module under test. Used extensively in **Bottom-Up Integration Testing** where high-level parent control modules have not yet been developed.
- **Stub:** A dummy called subroutine that simulates the interface and returns dummy values for sub-modules invoked by the module under test. Used extensively in **Top-Down Integration Testing** where low-level worker modules have not yet been developed.